# Project Specification: Intelligent Meeting, Decision & Action Tracking System

> **Objective:** Develop an intelligent organizational memory and action-tracking platform that improves accountability and ensures important decisions and follow-up actions are systematically captured, structured, and executed [cite: 1].

## 1. Core Features & Functional Requirements

The system must distinguish between general discussion, formal decisions, action items, and informational content [cite: 1].

### 1.1 Meeting Administration

- **Meeting Scheduling & Participant Management:** Interface to schedule meetings, invite attendees, and manage roles.
- **Agenda Management:** Define objectives prior to the meeting.
- **Meeting-Record Capture:** Upload audio/video or integrate live recording for processing.

### 1.2 Intelligent Extraction & Processing

- **Discussion Organization:** Segment raw transcripts into logical discussion blocks.
- **Decision Identification:** Automatically log formal decisions made during the meeting [cite: 1].
- **Action-Item Extraction:** Detect tasks, responsibility assignments, and deadline identification directly from speech [cite: 1].

### 1.3 Task Tracking & Workflow

- **Task Tracking:** A centralized board for tracking pending and completed items [cite: 1].
- **Automatic Reminders & Escalation:** Automated notifications for upcoming deadlines and escalation of overdue actions to managers [cite: 1].

### 1.4 Knowledge Base & Analytics

- **Decision History & Searchable Knowledge:** A semantic search engine to query past meetings and decisions [cite: 1].
- **Department-wise Dashboards:** Filtered views isolating tasks and metrics by department [cite: 1].
- **Meeting Effectiveness Analytics:** Metrics on closure rates, delays, and meeting productivity [cite: 1].

## 2. Technical Architecture & Tech Stack

### 2.1 AI & Extraction Pipeline (The Core Engine)

This pipeline is responsible for converting unstructured audio into structured data.

| Component | Recommended Technology | Purpose |
|---|---|---|
| Speech-to-Text | Whisper (faster-whisper) / Deepgram | Handles multi-speaker audio to generate accurate transcripts [cite: 1]. |
| Speaker Diarization | pyannote.audio | Identifies "who said what", crucial for automated responsibility assignment [cite: 1]. |
| Information Extraction | Llama 3.1/3.3 70B (via Groq) or Gemini Flash | Classifies segments into discussions, decisions, and action items [cite: 1]. |
| Structured Output | Pydantic + JSON Schema | Enforces the LLM to return clean JSON objects (owner, deadline, department) for database ingestion [cite: 1]. |

### 2.2 Backend & Data Storage

- **Relational Database: PostgreSQL** to store tasks, structured decisions, user profiles, and action item statuses [cite: 1].
- **Vector Database: ChromaDB** or **Qdrant** paired with sentence-transformers (all-MiniLM-L6-v2) to enable semantic natural-language search over past meeting transcripts and decisions [cite: 1].
- **Automation Engine: n8n** to handle chron-triggered workflows, check for overdue items daily, send reminders via Email/Slack, and execute escalation logic [cite: 1].

### 2.3 Frontend & User Interface

- **Framework: Next.js** (for a polished product feel) or **Streamlit** (for rapid prototyping) [cite: 1].
- **Data Visualization:** Utilize charting libraries (e.g., Recharts) to build meeting effectiveness analytics and department-wise dashboards [cite: 1].

## 3. System Workflow (Data Flow)

The standard operating procedure for the application involves the following sequential flow [cite: 1]:

1. **Ingestion:** Audio is uploaded and passed through Whisper (transcription) and pyannote (diarization) [cite: 1].

2. **Processing:** The combined transcript is fed into the LLM extraction layer to classify text into structured action items and decisions [cite: 1].
3. **Validation & Storage:** Pydantic validates the JSON output. Structured data goes to PostgreSQL, while text embeddings go to ChromaDB [cite: 1].
4. **Automation:** n8n monitors PostgreSQL continuously for deadlines, triggering notifications and escalations as needed [cite: 1].
5. **Presentation:** Users interact with the Next.js/Streamlit frontend to view dashboards, search the knowledge base, and update task statuses [cite: 1].

## 4. Implementation Priorities

For initial development, prioritize the core intelligence over tertiary features [cite: 1]:

- **High Priority:** Reliable transcription, LLM-based structured extraction (the core value proposition), basic dashboarding, and n8n escalation workflows [cite: 1].
- **Secondary Priority:** Full speaker diarization (can be complex to tune perfectly) and complex knowledge graphs (NetworkX/Neo4j can be added later for deep traceability) [cite: 1].